

UNIDAD 2

EL LENGUAJE JAVA: ASPECTOS BÁSICOS

**Programación
CFGS DAW**

Adaptado por: José Miguel Blázquez
2023/2024

Licencia



Reconocimiento – NoComercial – CompartirIgual (by-nc-sa): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original. NOTA: Esta es una obra adaptada por José Miguel Blázquez, derivada de la obra original realizada por José Antonio Díaz-Alejo , Carlos Cacho, Raquel Torres, Lionel Tarazón y Fco. Javier Valero.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:

 Importante

 Atención

 Interesante

ÍNDICE DE CONTENIDOS

1. Introducción – Primer ejemplo.....	4
2. Elementos básicos.....	7
2.1 Comentarios.....	7
2.2 Identificadores.....	7
3. Tipos de datos.....	9
4. Variables.....	10
4.1 Ámbito de una variable.....	12
4.2 Variables locales.....	12
4.3 Constantes (final).....	13
4.4 Reglas para nombrar variables.....	13
5. Operadores.....	14
5.1 Operadores aritméticos.....	14
5.2 Operadores relacionales.....	15
5.3 Operadores lógicos.....	16
5.4 Operador de asignación.....	17
5.5 Expresiones.....	17
5.6 Precedencia de operadores.....	18
5.7 La clase Math.....	19
6. Literales.....	20
6.1 Literales lógicos.....	20
6.2 Literales enteros.....	20
6.3 Literales reales.....	21
6.4 Literales carácter.....	21
6.5 Literales de tipo cadena de caracteres.....	22
7. Salida y entrada estándar.....	23
7.1 Salida estándar.....	23
7.2 Salida formateada.....	24
7.3 Entrada estándar.....	25
7.4 Ejemplo completo de E/S combinadas.....	29
7.5 Conversión entre tipos de datos.....	31
8. Aspectos técnicos de la plataforma Java.....	33
8.1 Transformación de código Java.....	33
8.2 Conceptos de la arquitectura Java: JDK, JRE, JVM, SE, EE, ME.....	34
8.3 Curiosidades sobre Java.....	35
8.4 Tipos de aplicaciones en Java.....	35

UD2. EL LENGUAJE JAVA: ASPECTOS BÁSICOS

1. INTRODUCCIÓN – PRIMER EJEMPLO

Java es un lenguaje de programación de propósito general y orientado a objetos que fue diseñado específicamente para tener tan pocas dependencias de implementación como fuera posible. Su objetivo es permitir que los desarrolladores de aplicaciones escriban el programa una vez y lo ejecuten en cualquier dispositivo (conocido en inglés como *WORA*, o "write once, run anywhere"), lo que quiere decir que el código puede escribirse una sola vez y ser ejecutado en cualquier tipo de dispositivos (PC, móvil, etc.).

Java tiene su propia máquina virtual para ejecutar los programas, por lo que las aplicaciones son independientes de la plataforma y podemos ejecutarlas en Linux, Windows, Mac y otros sistemas. De hecho, Java fue concebido para programar varios tipos de dispositivos electrónicos, incluidos los electrodomésticos; pero se hizo tan popular que rápidamente se centró en las aplicaciones informáticas.

Con Java y algunos otros lenguajes de esos años, empezamos a despedirnos de esta "puerta abierta" al mundo de bajo nivel. El acceso a la memoria y al sistema es mucho más restringido, y el lenguaje proporciona otras funcionalidades de alto nivel, como el recolector de basura, que se encarga de limpiar la memoria periódicamente, eliminando todo elemento que ya no se utiliza. Esta tarea era completamente manual en lenguajes anteriores (C o C++).

Java tiene hoy en día un amplio campo de aplicación: podemos desarrollar aplicaciones de escritorio con librerías como JavaFX, aplicaciones móviles (Android usa Java), aplicaciones web (con servlets y páginas JSP, o incluso con frameworks más avanzados, como Spring), etc.

Las características de Java son:

- Sencillo: Es un lenguaje "sencillo" de aprender.
- Orientado a Objetos: Posiblemente sea el lenguaje más orientado a objetos de todos los existentes; en Java, a excepción de los tipos fundamentales de variables (int, char, long...), todo es un objeto.
- Distribuido: Java está muy orientado al trabajo en red, soportando protocolos como TCP/IP, UDP, HTTP y FTP. Por otro lado el uso de estos protocolos es bastante sencillo comparándolo con otros lenguajes que los soportan.
- Robusto: El compilador Java detecta muchos errores que otros compiladores solo

detectarían en tiempo de ejecución o incluso nunca.

- Seguro: Sobre todo un tipo de desarrollo: los Applet. Estos son programas diseñados para ser ejecutados en una página web.
- Portable: En Java no hay aspectos dependientes de la implementación, todas las implementaciones de Java siguen los mismos estándares en cuanto a tamaño y almacenamiento de los datos.
- Arquitectura Neutral: El código generado por el compilador Java es independiente de la arquitectura: podría ejecutarse en un entorno UNIX, Mac, Windows, Móvil, etc.
- Rendimiento medio: Actualmente la velocidad de procesamiento del código Java es semejante a las de otros lenguajes orientados a objetos.
- Multithread: Soporta de modo nativo los threads (hilos de ejecución), sin necesidad del uso de librerías específicas.

La aplicación más pequeña posible es la que simplemente imprime un mensaje en la pantalla. Tradicionalmente, el mensaje suele ser "Hola Mundo!". Esto es justamente lo que hace el siguiente fragmento de código:

```
12 public class HolaMundo {
13
14     public static void main(String[] args) {
15         // TODO code application logic here
16         System.out.println("Hola Mundo!");
17     }
18
19 }
```

Hay que ver en detalle la aplicación anterior, línea a línea. Esas líneas de código contienen los componentes mínimos para mostrar la frase *Hola Mundo!* en la pantalla. Es un ejemplo muy simple, que no instancia objetos de ninguna otra clase; sin embargo, accede a otra clase incluida en el JDK.

public class HolaMundo

Esta línea **declara la clase *HolaMundo***. El nombre de la clase especificado en el fichero fuente **se utiliza para crear un fichero *nombredeclase.class*** en el directorio en el que se compila la aplicación. En este caso, el compilador creará un fichero llamado *HolaMundo.class*.

public static void main(String args[])

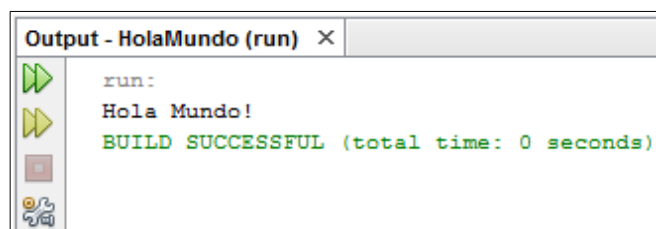
Esta línea **especifica un método** que el intérprete Java busca para ejecutar en primer lugar. Igual que en otros lenguajes, Java utiliza la **palabra clave *main*** para **especificar la primera función a ejecutar**. En Java, este bloque principal siempre debe ser **public, static y void**, y debe tener un

conjunto de parámetros de tipo `String[]` . Más adelante aprenderemos qué significa todo esto. De nuevo, todo el código perteneciente a este bloque principal debe estar encerrado entre sus correspondientes llaves.

```
System.out.println( "Hola Mundo!" );
```

Esta es la funcionalidad de la aplicación. Esta línea muestra el uso de un nombre de clase y método. Se usa el **método `println()`** de la **clase `out`** que está en el **paquete `System`**. El método `println()` toma una cadena como argumento y la escribe en el stream de salida estándar; en este caso, la ventana donde se lanza la aplicación. La clase `PrintStream` tiene un método instanciable llamado `println()`, que lo que hace es presentar en la salida estándar del Sistema el argumento que se le pase. En este caso, se utiliza la variable o instancia de `out` para acceder al método.

El resultado sería el siguiente:



⚡ Todas las instrucciones (creación de variables, llamadas a métodos, asignaciones) se deben **finalizar** con un **punto y coma**.

Cada clase pública debe tener el mismo nombre que el archivo fuente que la contiene, por lo que debemos almacenar el código fuente del ejemplo anterior en un archivo llamado `HolaMundo.java`. Si queremos compilar este código, usamos **`javac`** (la herramienta de nuestra instalación de JDK). Podemos hacerlo a través de cualquier IDE, como Visual Studio Code, Geany, o IntelliJ, siempre y cuando tengamos Java JDK correctamente instalado.

Luego, se generará un nuevo archivo llamado `HolaMundo.class`; este es el archivo compilado que se puede ejecutar bajo la Máquina Virtual Java (JVM), usando el comando **`java`**. Este último paso también se puede realizar bajo cualquier IDE. Después de ejecutar este programa, veremos un mensaje de "Hola mundo!" en la pantalla.

2. ELEMENTOS BÁSICOS

2.1 Comentarios

En Java hay tres tipos de comentarios:

```
// comentarios para una sola línea
```

```
/*  
comentarios de una o más líneas  
*/
```

```
/** comentario de documentación, de una o más líneas  
*/
```

Los dos primeros tipos de comentarios son los que todo programador conoce y se utilizan del mismo modo. Los **comentarios de documentación**, colocados inmediatamente antes de una declaración (de variable o función), indican que ese comentario ha de ser colocado en la documentación que se genera automáticamente cuando se utiliza la herramienta de Java, **javadoc**, no disponible en otros lenguajes de programación. Este tipo de comentario lo veremos más adelante.

2.2 Identificadores

Los **identificadores sirven para dar nombres a variables, funciones, clases y objetos**; cualquier cosa que el programador necesite identificar o usar.

Reglas para la creación de identificadores:

- **Java hace distinción entre mayúsculas y minúsculas**, por lo tanto, nombres o identificadores como var1, Var1 y VAR1 son distintos.
- Pueden estar formados por cualquiera de los caracteres del código Unicode, por lo tanto, se pueden declarar variables con el nombre: añoDeCreación, raím, etc., aunque eso sí, el **primer carácter no puede ser un dígito numérico** y **no pueden utilizarse espacios en blanco** ni símbolos coincidentes con operadores. ****RECOMENDACIÓN: usar char ingleses****
- La **longitud** máxima de los identificadores es prácticamente **ilimitada**.
- **No puede ser una palabra reservada del lenguaje** ni los valores lógicos true o false.
- **No pueden ser iguales a otro identificador** declarado en el mismo **ámbito**.

- **IMPORTANTE** (por convenio):
 - Los **nombres** de las **variables** y los **métodos** deberían **empezar** por una **letra minúscula** y los de las **clases** **por mayúscula**.
 - Si el identificador está formado por **varias palabras**, la **primera** se escribe en **minúsculas** (excepto para las clases) y el **resto** de palabras se hace **empezar por mayúscula** (por ejemplo: añoDeCreación).
 - Estas **reglas** deben seguirse de forma obligatoria en el módulo, ya que ayudan al proceso de codificación de un programa, así como a su legibilidad. Es más sencillo distinguir entre clases y métodos o variables.
 - Es recomendable ceñirse al alfabeto inglés por compatibilidad, es decir, evitar “ñ”, acentos, “ç”.
 - La longitud debe ser lo más reducida posible.

Serían identificadores válidos, por ejemplo:

contador	suma	edad	sueldoBruto	sueldoNeto
nombreUsuario	nombreCompleto	letraDni		

y su uso sería, por ejemplo:

```
int contador;           // crea variable de tipo int llamada contador
float sueldoNeto;       // crea variable de tipo float llamada sueldoNeto
char letraDni;         // crea variable de tipo char llamada letraDni
```


3. TIPOS DE DATOS

En Java existen 2 tipos **principales** de datos:

- **Tipos de datos simples:** Nos permiten crear variables que almacenan un solo valor. Por ejemplo para un contador, edad, precio, etc. Son los que más vamos a utilizar por ahora.
- **Tipos de datos compuestos:** Estructuras de datos más complejas que permiten almacenar muchos datos (vectores, objetos, etc.). Las veremos en futuras unidades.

Tipos de datos simples soportados por Java:

- Para números enteros: **byte, short, int, long**
- Para números reales (decimales): **float, double**
- Para caracteres: **char**
- Para valores lógicos: **boolean**.

Tipo	Descripción	Memoria ocupada	Rango de valores permitidos
byte	Número entero de 1 byte	1 byte	-128 ... 127
short	Número entero corto	2 bytes	-32768 ... 32767
int	Número entero	4 bytes	-2.147.483.648 ... 2.147.483.647
long	Número entero largo	8 bytes	-9223372036854775808 ... 9223372036854775807
float	Número real en coma flotante de precisión simple	4 bytes (32 bits)	$[\pm 3,4 \cdot 10^{-38} .. \pm 3,4 \cdot 10^{38}]$
double	Número real en coma flotante de precisión doble	64 bits	$[\pm 1,7 \cdot 10^{-308} ... \pm 1,7 \cdot 10^{308}]$
char	Un solo carácter	2 bytes	['\u0000' .. '\uffff'] o [0 .. 65.535] Unicode 16 bits
boolean	Valor lógico	1 bit	true o false

⚡ **Java no realiza una comprobación de los rangos.**

Por ejemplo: si a una variable de tipo short con el valor 32.767 se le suma 1, sorprendentemente el resultado será -32.768 (no produce un error de tipo **desbordamiento** como en otros lenguajes de programación, sino que se comporta de forma cíclica).

Existe un tipo de dato compuesto llamado **String** que conviene conocer ya que permite representar texto. Más adelante veremos cómo se utiliza.

4. VARIABLES

Las variables son elementos esenciales en todo programa, ya que nos permiten almacenar valores en ellas, para que podamos operar o modificarlas a lo largo de la ejecución del programa. Cada variable tiene un tipo asociado que nos permite saber qué tipo de información podemos almacenar en ella (ver apartado anterior). Siempre que queramos usar una variable ésta debe estar previamente declarada. Este paso consiste en hacer 2 cosas:

- a) Especificar el tipo de datos de la variable (entero, texto...)
- b) Especificando el nombre de la variable, que también se conoce como identificador de variable.

La forma básica de declarar (crear) una variable es la siguiente: **tipo identificador;**

Por ejemplo, creamos una variable de tipo int llamada edad: **int miVariable;**

Las variables pueden ser inicializadas en el momento de su declaración, es decir, se les puede dar un valor inicial al crearlas; o también se les puede dar valor posteriormente. Por ejemplo, creamos 2 variables de tipo int, a una le asignamos 3 como valor inicial, y a otra le asignaremos el 5 más adelante:

```
int miVariable = 3;
int miOtraVariable;
[...]
miOtraVariable = 5;
```

También es posible declarar varias variables en una sola línea. Por ejemplo, creamos tres variables de tipo float llamadas precio1, precio2 y precio3:

```
float precio1, precio2, precio3;
```

Esto es equivalente a:

```
float precio1;
float precio2;
float precio3;
```

A su vez, también pueden inicializarse en la creación. Por ejemplo:

```
float precio1 = 7.0, precio2 = 7.25, precio3 = 0.5;
```

Esto es equivalente a:

```
float precio1 = 7.0;
float precio2 = 7.25;
float precio3 = 0.5;
```

En resumen la declaración de variables sigue el siguiente patrón:

 **tipo** identificador [= valor][, identificador2 [= valor2] ...];

Es decir, es **obligatorio indicar el tipo y el identificador** (además de terminar en punto y coma como todas las instrucciones). Opcionalmente (indicado entre corchetes) se puede inicializar y/o se pueden declarar más variables.

⚡ Si una variable no ha sido inicializada, Java le asigna un valor por **defecto**.

Este valor es:

- Para las variables de tipo numérico es cero (0).
- Las variables de tipo char, el valor '\u0000'.
- Las variables de tipo boolean, el valor false.
- Para las variables de tipo referencial (objetos), el valor null.

Es una **buena práctica inicializar siempre todas las variables**.

Palabras clave: las siguientes son palabras clave que no se pueden utilizar como identificadores ya que Java las utiliza para otras cosas:

abstract	continue	for	new	switch
boolean	default	goto	null	synchronized
break	do	if	package	this
byte	double	implements	private	threadsafe
byvalue	else	import	protected	throw
case	extends	instanceof	public	transient
catch	false	int	return	true
char	final	interface	short	try
class	finally	long	static	void
const	float	native	super	while

Palabras reservadas: el lenguaje se reserva unas cuantas palabras más, pero que hasta ahora no tienen un cometido específico. Son:

cast	uture	generic	inner
operator	outer	rest	var

4.1 Ámbito de una variable

 El **ámbito** de una variable es la zona del programa donde dicha variable es visible y puede utilizarse.

El ámbito de una variable depende del lugar del programa donde es declarada, pudiendo pertenecer a cuatro categorías distintas:

1. Variable local.
2. Atributo de clase.
3. Parámetro de un método.
4. Parámetro de un manejador de excepciones.

Por ahora utilizaremos solo variables locales, las demás categorías las veremos en posteriores unidades.

4.2 Variables locales

 Una **variable local** se declara dentro del cuerpo de un método de una clase y es **visible únicamente dentro de dicho método**.

Se puede declarar en cualquier lugar del cuerpo, incluso después de instrucciones ejecutables, aunque es una **buena costumbre declararlas justo al principio**. También pueden declararse variables dentro de un bloque con llaves {...}. En ese caso, sólo serán “visibles” dentro de dicho bloque. Por ejemplo (No es necesario entender lo que hace el programa) :

```
14  public static void main(String[] args) {  
15  
16      int i;  
17  
18      for (i=0;i<10;i++)  
19          System.out.println(i);  
20  }
```

En este ejemplo existe una variable local: **int i**; únicamente puede utilizarse dentro del bloque **main** donde fué creada.

4.3 Constantes (final)

Al declarar una variable puede utilizarse la palabra reservada **final** para indicar que el valor de la variable no podrá modificarse (es una constante).

Por ejemplo, creamos variable constante tipo int llamada x con valor 18:

```
final int X = 18;
```

Por ejemplo, creamos variable constante tipo float llamada pi con valor 3.14:

```
final float PI = 3.14;
```

Si posteriormente intentamos modificar sus valores se producirá un error y Java nos avisará de que no es posible:

```
X = 20;           // no permitido, produce error  
PI = 7;          // no permitido, produce error
```

⚡ Se considera obligatorio **nombrar** las **constantes** con **mayúsculas**.

4.4 Reglas para nombrar variables

A la hora de nombrar un identificador existen una serie de normas de estilo de uso generalizado que, no siendo obligatorias, se usan en la mayor parte del código Java. Estas reglas para la nomenclatura de variables son las siguientes:

- **Java distingue las mayúsculas de las minúsculas.** Por ejemplo, `Alumno` y `alumno` son variables diferentes.
- **Comienzan por letra minúscula**, y si tienen más de una palabra se colocan juntas y el resto comenzando por mayúsculas o separadas por “_” (guión bajo). **Nunca espacios.**
- **No** se pueden utilizar literales booleanos (`true` o `false`) ni el valor nulo (`null`).
- Los **identificadores deben ser lo más descriptivos** posibles. Es mejor usar palabras completas en vez de abreviaturas crípticas. Así, nuestro código será más fácil de leer y comprender. Por ejemplo, si tenemos que darle el nombre a una variable que almacena los datos de un cliente sería recomendable que la misma se llamara algo así como `ficheroClientes` o `manejadorCliente`, y no algo poco descriptivo como `cli33`.

5. OPERADORES

Los **operadores** son una parte indispensable de la programación ya que nos permiten realizar cálculos matemáticos y lógicos, y evaluar expresiones, entre otras cosas. Los operadores principales se clasifican en estas categorías:

- Aritméticos: sumas, restas, multiplicaciones, etc.
- Relacionales: menor, menor o igual, mayor, mayor o igual, igual a, etc.
- Lógicos: and, or, not, etc.
- Bits: prácticamente no los utilizaremos en este curso.
- Asignación: =

5.1 Operadores aritméticos

Operador	Formato	Descripción
+	op1 + op2	Suma aritmética de dos operandos.
-	op1 - op2 -op1	Resta aritmética de dos operandos. Cambio de signo.
*	op1 * op2	Multiplicación de dos operandos
/	op1 / op2	División entera de dos operandos
%	op1 % op2	Resto de la división entera (o módulo)
++	++op1 op1++	Incremento unitario (pre y post)
--	--op1 op1--	Decremento unitario (pre y post)

En cuanto al operador de **división** debemos tener en cuenta que produce un resultado del mismo tipo que los números involucrados. En otras palabras, si dividimos dos números enteros, como 5/2, entonces el resultado será un número entero (2), no real (2,5). El operador de módulo obtiene el módulo (resto) de una división entera. En el ejemplo anterior, 5 % 2 obtiene el resto de dividir 5 entre 2, que es 1.

El operador - puede utilizarse en su versión unaria (- op1) y la operación que realiza es la de invertir el signo del operando.

Los operadores ++ y -- realizan un incremento y un decremento unitario respectivamente. Es decir:

x++ equivale a **x = x + 1**

x-- equivale a **x = x - 1**

Los operadores ++ y -- admiten notación **postfija** y **prefija**:

- **op1++ / op1--**: Primero se ejecuta la instrucción en la cual está inmerso y después se incrementa o decrementa op1.
- **++op1 / --op1**: Primero se incrementa o decrementa op1 y después ejecuta la instrucción en la cual está inmerso.

Los operadores incrementales suelen utilizarse en estructuras condicionales y en los bucles (estructuras repetitivas), aunque también pueden emplearse en expresiones normales.

5.2 Operadores relacionales

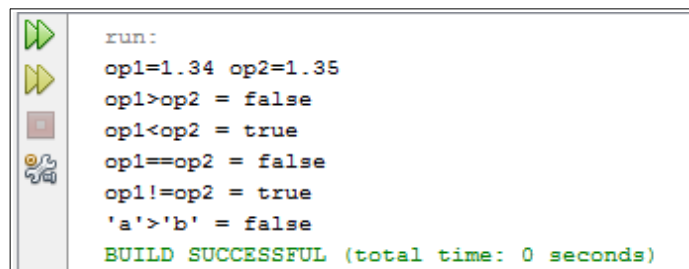
Operador	Formato	Descripción
>	op1 > op2	Devuelve true (cierto) si op1 es mayor que op2
<	op1 < op2	Devuelve true (cierto) si op1 es menor que op2
>=	op1 >= op2	Devuelve true (cierto) si op1 es mayor o igual que op2
<=	op1 <= op2	Devuelve true (cierto) si op1 es menor o igual que op2
==	op1 == op2	Devuelve true (cierto) si op1 es igual a op2
!=	op1 != op2	Devuelve true (cierto) si op1 es distinto de op2

Los operadores relacionales actúan sobre valores enteros, reales y caracteres (char); y devuelven un valor del tipo boolean (true o false).

Ejemplo:

```
15 public static void main(String[] args){
16
17     double op1,op2;
18     char op3,op4;
19
20     op1=1.34;
21     op2=1.35;
22     op3='a';
23     op4='b';
24
25     System.out.println("op1=" + op1 + " op2=" + op2);
26     System.out.println("op1>op2 = " + (op1 > op2));
27     System.out.println("op1<op2 = " + (op1 < op2));
28     System.out.println("op1==op2 = " + (op1 == op2));
29     System.out.println("op1!=op2 = " + (op1 != op2));
30     System.out.println("'a'>'b' = " + (op3 > op4));
31
32 }
```

Salida:



```
run:
op1=1.34 op2=1.35
op1>op2 = false
op1<op2 = true
op1==op2 = false
op1!=op2 = true
'a'>'b' = false
BUILD SUCCESSFUL (total time: 0 seconds)
```

5.3 Operadores lógicos

Operador	Formato	Descripción
&&	op1 && op2	Y lógico. Devuelve true (cierto) si son ciertos op1 y op2
	op1 op2	O lógico. Devuelve true (cierto) si son ciertos op1 o op2
!	! op1	Negación lógica. Devuelve true (cierto) si es false op1.

Estos operadores actúan sobre operadores o expresiones lógicas, es decir, aquellos que se evalúan a cierto o falso (true / false).

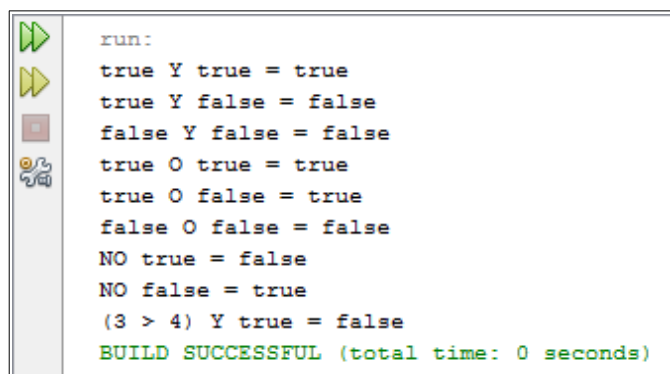
Ejemplo:

```

15 public static void main(String[] args){
16
17     boolean a, b, c, d;
18
19     a=true;
20     b=true;
21     c=false;
22     d=false;
23
24     System.out.println("true Y true = " + (a && b) );
25     System.out.println("true Y false = " + (a && c) );
26     System.out.println("false Y false = " + (c && d) );
27     System.out.println("true O true = " + (a || b) );
28     System.out.println("true O false = " + (a || c) );
29     System.out.println("false O false = " + (c || d) );
30     System.out.println("NO true = " + !a);
31     System.out.println("NO false = " + !c);
32     System.out.println("(3 > 4) Y true = " + ((3 >4) && a) );
33
34 }
35

```


Salida:



```
run:
true Y true = true
true Y false = false
false Y false = false
true O true = true
true O false = true
false O false = false
NO true = false
NO false = true
(3 > 4) Y true = false
BUILD SUCCESSFUL (total time: 0 seconds)
```

5.4 Operador de asignación

El operador simple de asignación es el símbolo igual: **=**

variable = expresión

La sintaxis anterior asigna a la **variable** el resultado de evaluar la **expresión** de la derecha. Es posible combinar el operador de asignación con otros operadores para, de forma **abreviada**, realizar un cálculo y asignarlo a una variable:

Operador	Formato	Equivalencia
+=	op1 += op2	op1 = op1 + op2
-=	op1 -= op2	op1 = op1 - op2
*=	op1 *= op2	op1 = op1 * op2
/=	op1 /= op2	op1 = op1 / op2
%=	op1 %= op2	op1 = op1 % op2

5.5 Expresiones

Una **expresión** es la combinación de varios operadores y operandos de cualquier tipo. Por ejemplo, veamos unos ejemplos des siguientes expresiones:

$7 + 5 * 4 - 2$

$10 + (1 \% 5)$

$(7 * x) < \text{precio}$

El lenguaje **Java** evalúa las expresiones aplicando los operadores uno a uno siguiendo un orden **específico**. Este orden se detalla en el siguiente punto.

5.6 Precedencia de operadores

Se refiere al **orden en el que se evalúan los operadores** en una expresión. No es necesario saberse toda la lista de memoria, pero **es imprescindible conocer al menos los más utilizados (en negrita)**: matemáticos, relacionales, lógicos y de asignación. Algunos de estos operadores los veremos en unidades posteriores, ahora mismo no es necesario que sepas qué hacen.

1	Operadores postfijos:	<i>[] . (paréntesis)</i>
2	Operadores unarios:	<i>++expr, --expr, -expr, !</i>
3	Creación o conversión de tipo:	<i>new (tipo)expr</i>
4	Multiplicación y división:	<i>*, /, %</i>
5	Suma y resta:	<i>+, -</i>
6	Desplazamiento de bits:	<i><<, >>, >>></i>
7	Relacionales:	<i><, >, <=, >=</i>
8	Igualdad y desigualdad:	<i>==, !=</i>
9	AND a nivel de bits:	<i>&</i>
10	AND lógico:	<i>&&</i>
11	XOR a nivel de bits:	<i>^</i>
12	OR a nivel de bits:	<i> </i>
13	OR lógico:	<i> </i>
14	Operador condicional:	<i>?:</i>
15	Asignación:	<i>=, +=, -=, *=, /=, %=</i>

5.7 La clase Math

Para poder utilizar operadores matemáticos más potentes en Java, se ha incluido una clase especial llamada **Math** dentro del paquete `java.lang`. Esta clase posee muchos métodos interesantes para realizar cálculos matemáticos complejos como el cálculo de potencias, raíces cuadradas, valores absolutos, seno, coseno, logaritmos, etc.

Por ejemplo:

```
double x = Math.pow(3,3);    // Potencia 3 ^ 3
double y = Math.sqrt(9);     // Raíz cuadrada de 9
```

También posee constantes definidas como estas:

```
double PI -> El número  $\Pi$  (3,14159265...)
double E -> El número e (2, 7182818245...)
```

Algunos ejemplos de otros métodos:

 E	double	^
 PI	double	
 IEEEremainder (double f1, double f2)	double	
 abs (double a)	double	
 abs (float a)	float	
 abs (int a)	int	
 abs (long a)	long	
 acos (double a)	double	
 addExact (int x, int y)	int	
 addExact (long x, long y)	long	
 asin (double a)	double	
 atan (double a)	double	
 atan2 (double y, double x)	double	
 cbrt (double a)	double	
 ceil (double a)	double	
 copySign (double magnitude, double sign)	double	
 copySign (float magnitude, float sign)	float	▼

6. LITERALES

A la hora de tratar con los valores de los tipos de datos simples (y Strings) se utiliza lo que se denomina “literales”. Los literales son elementos que sirven para representar un valor concreto en el código fuente del programa (es decir, un número, una letra....).

En Java existen literales para los siguientes tipos de datos:

- Lógicos (boolean).
- Carácter (char).
- Enteros (byte, short, int y long).
- Reales (double y float).
- Cadenas de caracteres (String).

6.1 Literales lógicos

Son únicamente 2, las palabras reservadas **true** y **false**.

6.2 Literales enteros

Los literales de tipo entero: *byte*, *short*, *int* y *long* pueden expresarse en notación decimal (base 10), octal (base 8) o hexadecimal (base 16). Además, puede añadirse al final del mismo la letra *L* para indicar que el entero es considerado como long (64bits).

En Java, el compilador identifica un entero decimal (base 10) al encontrar un número cuyo primer dígito es cualquier símbolo decimal excepto el cero (del 1 al 9). A continuación pueden aparecer dígitos del 0 al 9.

La letra *L* al final de un literal de tipo entero puede aplicarse a cualquier sistema de numeración e indica que el número decimal sea tratado como un entero largo (de 64 bits). Esta letra *L* puede ser mayúscula o minúscula, aunque es aconsejable utilizar la mayúscula ya que de lo contrario puede confundirse con el dígito uno (1).

Ejemplo:

```
long max1 = 9223372036854775807L; //valor máximo para un entero largo
```

6.3 Literales reales

Los literales de tipo real sirven para indicar valores *float* o *double*. A diferencia de los literales de tipo entero, **no pueden expresarse en octal o hexadecimal**. Existen dos formatos de representación: mediante su parte entera, el punto decimal (.) y la parte fraccionaria; o mediante notación exponencial o científica:

Ejemplos equivalentes:

3.1415	0.31415e1	.31415e1	0.031415E+2
.031415e2	314.15e-2	31415E-4	

Al igual que los literales que representan enteros se puede poner una letra como sufijo. Esta letra puede ser una *F* o una *D* (mayúscula o minúscula indistintamente).

F --> Trata el literal como de tipo *float*.

D --> Trata el literal como de tipo *double*.

Ejemplos: 3.1415F .031415d

ATENCIÓN: por defecto, Java interpreta los literales de tipo real como “double” si no especificamos nada al escribirlos.

6.4 Literales carácter

Los literales de tipo carácter se representan siempre entre comillas simples. Entre las comillas simples puede aparecer:

- Un **símbolo** (letra) siempre que el carácter esté asociado a un código Unicode.
Ejemplos: ‘a’, ‘B’, ‘{’, ‘ñ’, ‘á’.
- Una “**secuencia de escape**”. Las secuencias de escape son combinaciones del símbolo contrabarra \ seguido de una letra, y sirven para representar caracteres que no tienen una equivalencia en forma de símbolo gráfico.

Las secuencias de escape más habituales son:

Secuencia	Significado
\n	Nueva Línea.
\t	Tabulador
\"	Dobles comillas.
\'	Comilla simple.
\\	Barra invertida.

Por ejemplo:

Para imprimir (en pantalla) una diagonal inversa se utiliza: \\

Para imprimir (en pantalla) comillas dobles en un String se utiliza: \"

6.5 Literales de tipo cadena de caracteres

Los **Strings o cadenas de caracteres no forman parte de los tipos de datos elementales en Java**, sino que se consideran objetos más complejos y **son instanciados a partir de la clase `java.lang.String`**, pero aceptan su inicialización a partir de literales de este tipo, por lo que los vamos a presentar en este punto.

✎ Un literal de tipo String va encerrado entre comillas dobles (") y debe estar incluido completamente en una sola línea del programa fuente (no puede dividirse en varias líneas). (Hay una excepción indicada más abajo...)

Entre las comillas dobles pueden incluirse cualquiera de los carácter del código Unicode (o su código precedido del carácter \) además de las secuencias de escape vistas anteriormente en los literales de tipo carácter. Así, por ejemplo, para incluir un salto de línea dentro de un literal de tipo String deberá hacerse mediante la secuencia de escape \n :

Ejemplo:

```
System.out.println("Primera línea\nSegunda línea del String\n");  
System.out.println("Hola");
```

La ejecución *del programa* anterior produciría la siguiente salida por pantalla:

Primera línea

Segunda línea del String

Hola

La forma de incluir los caracteres: comillas dobles (") y contrabarra (\) es mediante las secuencias de escape \" y \\ respectivamente (o mediante su código Unicode precedido de \).

🔊 Si el string es demasiado largo y debe dividirse en varias líneas en el fichero fuente, puede utilizarse el operador de concatenación de strings (+) de la siguiente forma:

```
"Este String es demasiado largo para estar en una línea del " +  
"fichero fuente y se ha dividido en dos."
```

7. SALIDA Y ENTRADA ESTÁNDAR

Ahora vamos a aprender en detalle a interactuar con el usuario final. En primer lugar, veremos cómo mostrar valores en la pantalla usando diferentes instrucciones, y luego veremos cómo recopilar información del teclado y convertirla al tipo de datos apropiado.

7.1 Salida estándar

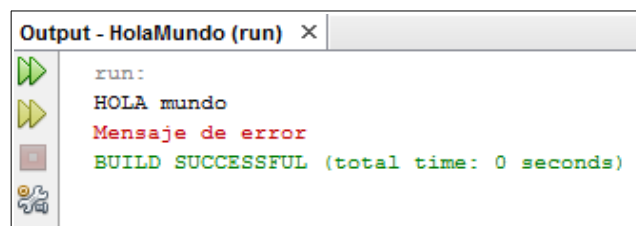
Ya hemos visto el uso de **System.out** para mostrar información por pantalla y uno de estos 2:

- **print("...")** imprime texto por pantalla.
- **println("...")** imprime texto por pantalla e introduce un salto de línea.

La utilización de *System.err* sería totalmente análoga para enviar los mensajes producidos por errores en la ejecución (es el canal que usa también el compilador para notificar los errores encontrados). Por ejemplo, para presentar el mensaje de saludo habitual por pantalla, y después un mensaje de error, tendríamos la siguiente clase (aunque en realidad toda la información va a la consola de comandos donde estamos ejecutando el programa):

```
14 public static void main(String[] args) {  
15  
16     System.out.print("HOLA ");  
17     System.out.println("mundo");  
18     System.err.println("Mensaje de error");  
19 }
```

Y la salida sería la siguiente:



```
Output - HolaMundo (run) ×  
run:  
HOLA mundo  
Mensaje de error  
BUILD SUCCESSFUL (total time: 0 seconds)
```

También pueden imprimirse variables de cualquier tipo, así como combinaciones de texto y variables concatenadas con el operador +

```
14 public static void main(String[] args) {  
15     String nombre = "Pepito";  
16     int edad = 25;  
17     System.out.println(nombre);  
18     System.out.println(edad);  
19     System.out.println(nombre + " tiene " + edad + " años");  
20 }
```

Y la salida sería la siguiente:

```
Pepito  
25  
Pepito tiene 25 años.
```

7.2 Salida formateada

Además de las 2 instrucciones tradicionales para imprimir datos, podemos usar algunas otras opciones si queremos que estos datos tengan un formato de salida determinado. Para ello, podemos utilizar la instrucción “**printf**” en lugar de las anteriores. Por ejemplo, si usamos esta instrucción:

```
System.out.printf("The number is %d", number);
```

El símbolo %d será reemplazado por el contenido de la variable number, y esta variable debe ser un número entero (esto es lo que significa %d).

Hay otros símbolos para representar diferentes tipos de datos. Éstos son algunos de ellos:

- %d para tipos enteros (long, int)
- %f para tipos reales (float, double)
- %s para cadenas
- %c para caracteres

Además de los símbolos primarios %d y %f, podemos agregar más otra información entre el '%' y la letra, para especificar información de formato.

Especificación de dígitos enteros

Si queremos generar un número entero con un número determinado de dígitos, podemos hacerlo de esta manera:

```
System.out.printf("The number is %05d", number);
```

donde 05 significa que el número entero va a tener, al menos, 5 dígitos, y si no hay suficientes dígitos en el número, entonces se rellenará con ceros. La salida de esta instrucción si el número es 33 sería “**The number is 00033**”. Si no ponemos el 0, entonces el número se llenará de espacios en blanco. Así que esta instrucción:

```
System.out.printf("The number is %10d", number);
```

siendo “number” un 33, produciría el siguiente resultado: “**The number is 33**”.

Especificación de dígitos de fracción

De la misma manera que formateamos números enteros, podemos formatear números reales, aspecto que suele ser más habitual y necesario. Podemos usar el mismo patrón visto antes para

especificar el número total de dígitos enteros:

```
System.out.printf("The number is %3f", number);
```

Pero, además, podemos especificar el número total de dígitos de la parte fraccionaria añadiendo un punto y la cantidad de dígitos deseados, de esta forma:

```
System.out.printf("The number is %3.3f", number);
```

Entonces, si la variable “number” fuera 3.14159, la salida sería “**The number is 3.142**”.

7.3 Entrada estándar

La entrada estándar (leer información del teclado, escrita por el usuario) es un poco más compleja. Hay varias formas de hacerlo pero la más sencilla es utilizar la **clase Scanner**.

Siempre que queramos leer información del teclado primero tendremos que declarar un objeto Scanner que nos permita leer de la entrada estándar “System.in” así:

```
Scanner reader = new Scanner(System.in);
```

*NOTA: En este ejemplo hemos creado un objeto Scanner llamado reader pero podríamos ponerle **cualquier** nombre en vez de reader, como por ejemplo “sc”, “teclado”, etc...*

Ahora utilizando a “reader” podremos acceder al teclado tantas veces como queramos para leer información. Por ejemplo:

```
String texto = reader.nextLine();
```

El método **.nextLine()** capturará el texto que el usuario escriba por teclado (hasta presionar la tecla Intro) y lo guardará en la variable **texto** (de tipo String).

Existen mucho otros métodos según el tipo de dato que se quiera leer:

- **nextByte()**: obtiene un número entero tipo byte.
- **nextShort()**: obtiene un número entero tipo short.
- **nextInt()**: obtiene un número entero tipo int.
- **nextLong()**: obtiene un número entero tipo long.
- **nextFloat()**: obtiene un número real float.
- **nextDouble()**: obtiene un número real double.
- **next()**: obtiene el siguiente token (texto hasta un espacio).

⚡ No existen métodos de la clase Scanner para obtener directamente booleanos **ni para obtener un solo carácter (leer una letra)**. Hay que simularlo de otras formas.

IMPORTANTE: Para poder utilizar la clase Scanner es necesario importarla desde el paquete java.util de Java. Es decir, arriba del todo (antes del public class...) hay que escribir la siguiente sentencia:

```
import java.util.Scanner;
```

Cuidado al combinar tipos de datos

Supongamos que hay que leer esta información de la entrada estándar:

```
23 43
Hello world
```

Podemos pensar que necesitamos usar el método nextInt 2 veces y luego el método nextLine para leer la cadena, pero este enfoque NO es correcto: cuando se usa nextInt para leer los valores enteros, éste método no captura el final de la línea que existe más allá del número 43, entonces, cuando usa el método nextLine una vez, solo lee el salto de línea, pero no captura el texto de la segunda línea. La secuencia correcta sería esta:

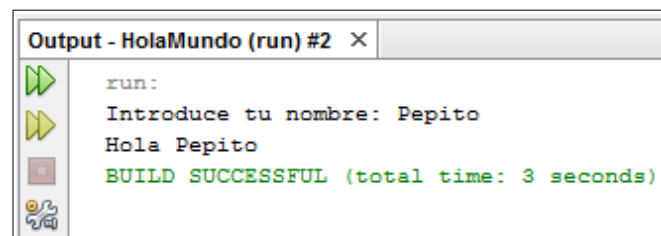
```
int number1 = sc.nextInt();
int number2 = sc.nextInt();
sc.nextLine();
String text = sc.nextLine();
```

La tercera línea lee y descarta (no almacena) el carácter del final del “43” (salto de línea).

Ejemplo en el que leemos una cadena de texto y la mostramos por pantalla:

```
12  import java.util.Scanner;
13
14  public class EjemploScanner {
15
16      public static void main(String[] args){
17
18          String nombre;
19
20          Scanner entrada = new Scanner(System.in);
21
22          System.out.print("Introduce tu nombre: ");
23
24          nombre = entrada.nextLine();
25
26          System.out.println("Hola " + nombre);
27
28      }
29  }
```

Salida:

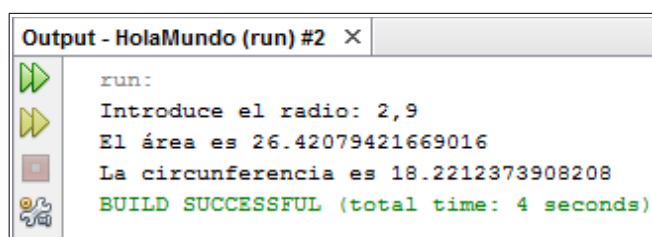


```
Output - HolaMundo (run) #2 X
run:
Introduce tu nombre: Pepito
Hola Pepito
BUILD SUCCESSFUL (total time: 3 seconds)
```

Ejemplo en el que leemos un valor tipo double. El programa pide al usuario que introduzca el radio de un círculo, luego calcula su área y circunferencia, por último lo muestra por pantalla.

```
12 import java.util.Scanner;
13
14 public class EjemploScanner {
15
16     public static void main(String[] args){
17
18         double radio, area, circunferencia;
19
20         Scanner entrada = new Scanner(System.in);
21
22         System.out.print("Introduce el radio: ");
23
24         radio = entrada.nextDouble();
25
26         // Se hace uso de la librería Math para usar PI y la portencia(pow)
27         area = Math.PI * Math.pow(radio, 2);
28
29         circunferencia = 2 * Math.PI * radio;
30
31         System.out.println("El área es " + area);
32
33         System.out.println("La circunferencia es " + circunferencia);
34     }
35 }
```

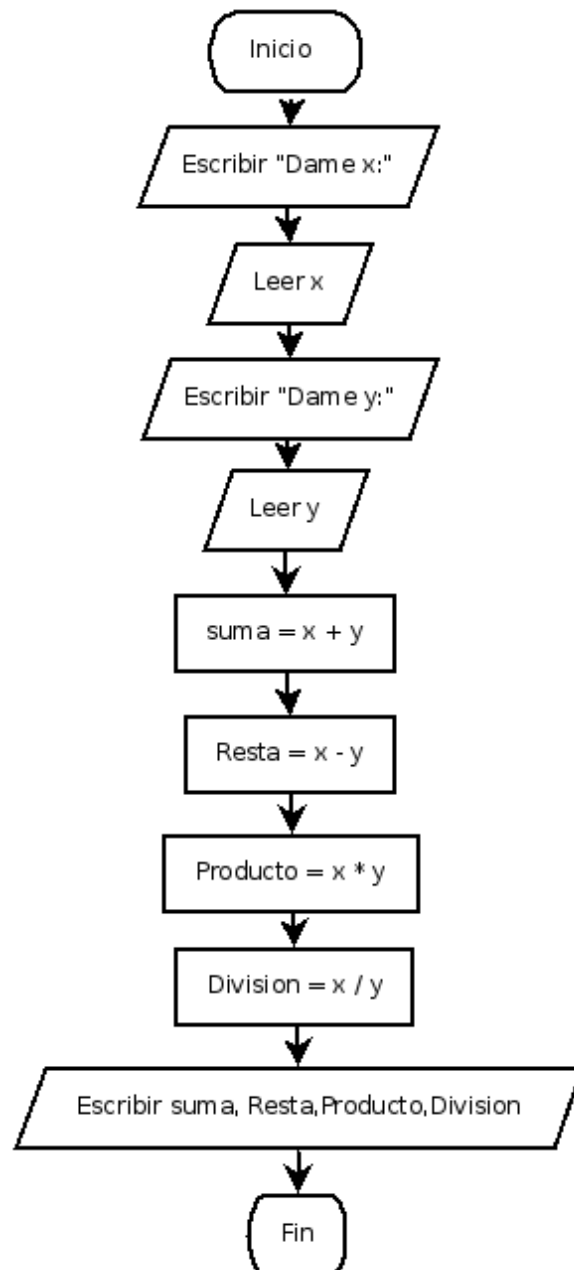
Salida:



7.4 Ejemplo completo de E/S combinadas

Programa que lea 2 números, calcule y muestre el valor de su suma, resta, producto y división.

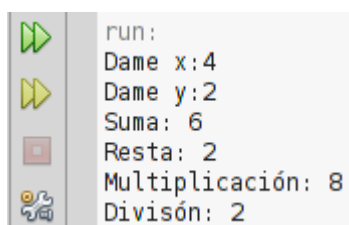
Diagrama de flujo:



Código:

```
1  package ejemplo1;
2
3  import java.util.Scanner; // Importamos la clase Scanner
4
5  public class Ejemplo1 {
6
7      public static void main(String[] args) {
8
9          // Declaramos las variables que vamos a necesitar
10         int x, y, suma, resta, mult, div;
11
12         // Creamos el objeto Scanner para leer por teclado
13         Scanner reader = new Scanner(System.in);
14
15         // Pedimos y leemos x
16         System.out.print("Dame x:");
17         x = reader.nextInt();
18
19         // Pedimos y leemos y
20         System.out.print("Dame y:");
21         y = reader.nextInt();
22
23         // Realizamos los cálculos necesarios
24         suma = x + y;
25         resta = x - y;
26         mult = x * y;
27         div = x / y;
28
29         // Mostramos los cálculos por pantalla
30         System.out.println("Suma: " + suma);
31         System.out.println("Resta: " + resta);
32         System.out.println("Multiplicación: " + mult);
33         System.out.println("División: " + div);
34     }
35 }
```

Salida:



```
run:
Dame x:4
Dame y:2
Suma: 6
Resta: 2
Multiplicación: 8
División: 2
```

7.5 Conversión entre tipos de datos

A veces necesitamos convertir un valor de algún tipo en otro tipo diferente. La forma en que hacemos este paso depende de los tipos involucrados.

Las conversiones entre **valores numéricos** son bastante sencillas. Solo nos falta hacer un **typecast**, es decir, especificar entre paréntesis el tipo de dato al que queremos convertir la expresión. En este ejemplo, estamos convirtiendo el valor real de “pi” en un número entero (por lo que obtenemos un valor 3 como resultado final):

```
float pi = 3.1416f;
int piInteger = (int)pi;
```

También se puede hacer el paso contrario. En este caso, estamos convirtiendo un valor entero en uno decimal (el valor final será 5,0):

```
int number = 5;
double realNumber = (double)number;
```

Sin embargo, este paso NO es necesario si el tipo de origen es más pequeño que el tipo de destino. Por ejemplo, un byte no necesita convertirse en int:

```
byte value = 3;
int number = value;
```

El casting de tipos (typecast) puede ser útil, por ejemplo, para convertir divisiones enteras en divisiones reales. Este ejemplo divide dos valores enteros, pero, como estamos convirtiendo uno de ellos en float, entonces el resultado final será un número real, con los dígitos de fracción correspondientes, y se puede almacenar en una variable float:

```
float result = (float) 3 / 2;
```

Otra forma de hacerlo sin typecasting:

```
float result = 3 / 2.0f;           //¿qué pasaría si quitáramos la “f” del final del 2.0?
```

En general, toda operación aritmética trata de producir un resultado del mismo tipo que sus operandos (si dividimos dos enteros, obtenemos un resultado entero). Sin embargo, en ciertas operaciones como sumas o multiplicaciones, Java intenta convertir el resultado a un tipo superior y necesitamos “encasillarlo”. En este ejemplo, tratamos de sumar valores de dos bytes, pero Java intenta convertir el resultado de la operación a int, por lo que debemos especificar que queremos seguir usando byte como resultado (aunque podríamos causar un desbordamiento):

```
byte a = 3, b = 2;
byte result = (byte)(a + b);
```

Si mezclamos 2 tipos diferentes en una operación aritmética, Java convierte el resultado al de mayor precisión de ellos. Esta multiplicación obtiene un número decimal (float) porque uno de los operandos es float:

```
float a = 3.5f;  
int b = 4;  
float result = a * b;
```

Conversión de cadena ó a cadena

En algunas situaciones, podemos leer valores numéricos de fuentes textuales, como archivos de texto o entradas del usuario. En este caso, necesitamos convertir el texto en el valor numérico correspondiente. Para hacer esto, Java proporciona algunas instrucciones útiles. Aquí puedes ver las más importantes:

- **Integer.parseInt** convierte un valor de texto en int:

```
int value = Integer.parseInt("23");
```

- **Float.parseFloat**, **Double.parseDouble**, **Byte.parseByte**, **Short.parseShort** y **Long.parseLong** hacen lo mismo con sus tipos de datos correspondientes:

```
float value = Float.parseFloat("3.1416");
```

Si queremos convertir un valor numérico en una cadena (String) podemos elegir una de estas soluciones:

- Concatenar el valor numérico con una cadena vacía "":

```
int number = 23;  
String text = "" + number;
```

- Utilizar la instrucción **String.valueOf** para convertir el valor especificado en una cadena:

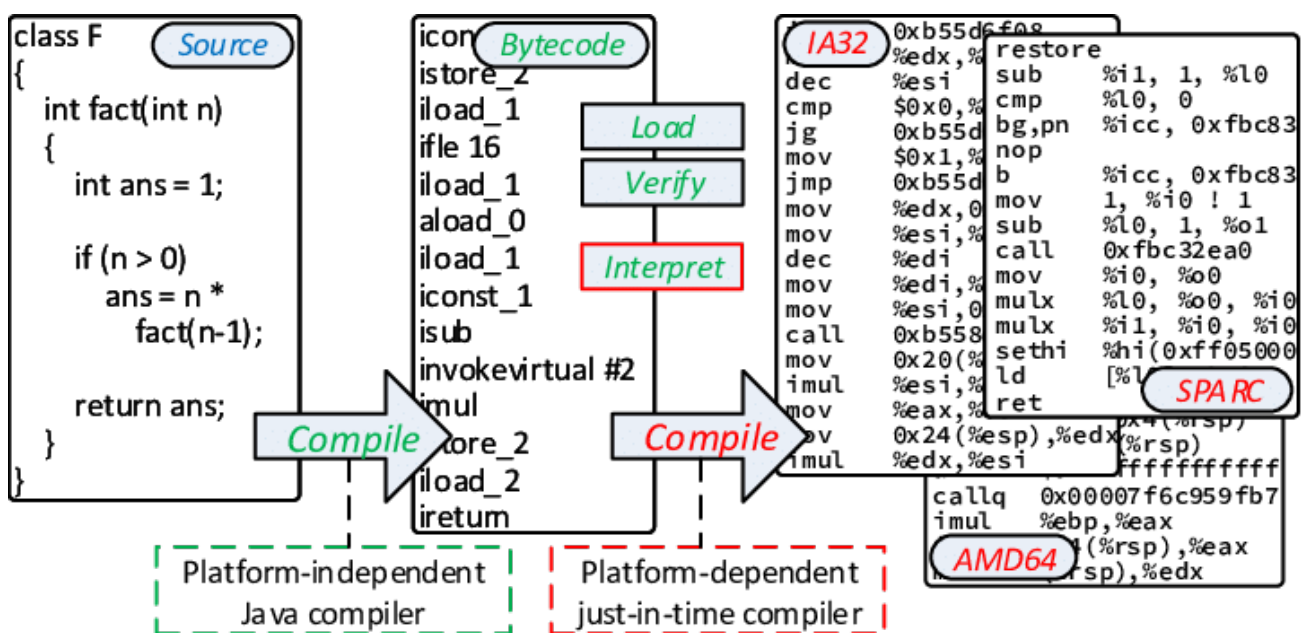
```
int number = 23;  
String text = String.valueOf(number);
```


8. ASPECTOS TÉCNICOS DE LA PLATAFORMA JAVA

8.1 Transformación de código Java

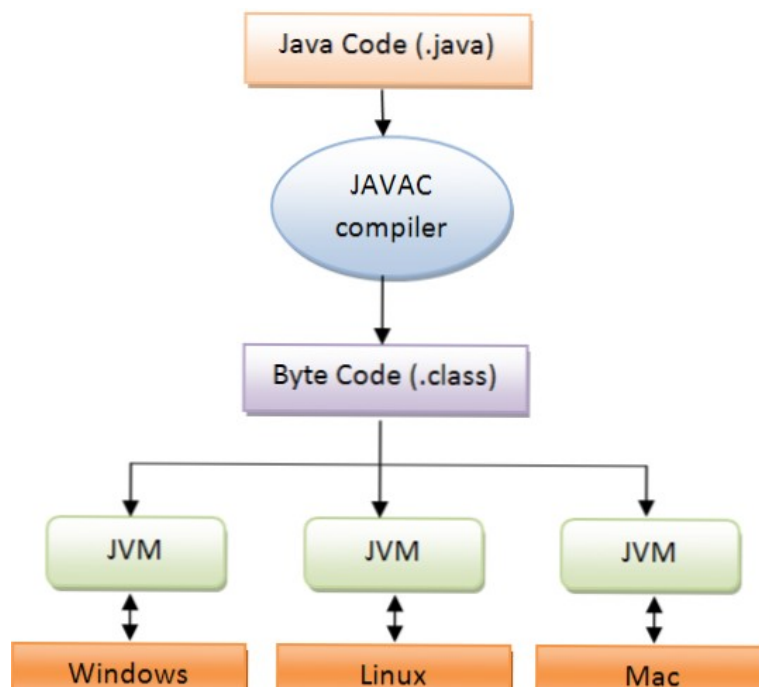
En la siguiente imagen podemos observar cómo el código que escribimos (Source) se transforma en primer lugar a “Bytecode” mediante un compilador independiente de la plataforma. Tras este paso, un intérprete se encargará de traducir el bytecode a código máquina ejecutable en la plataforma correspondiente.

El intérprete sí depende de la plataforma de ejecución, el cual denominamos “Máquina Virtual Java” (JVM): Java Virtual Machine.



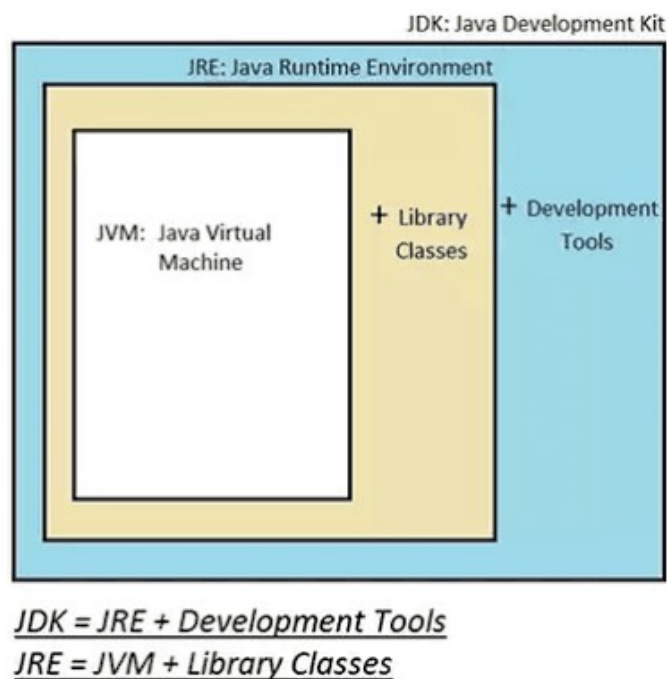
Esto permite escribir código en un entorno Windows y ejecutarlo en un entorno Linux, y viceversa.

La siguiente imagen ilustra cómo a partir de nuestro fichero de código fuente (.java), el compilador genera un bytecode (.class). Tras esto, según la JVM utilizada, ése programa podrá ser ejecutado (interpretado) en diversas plataformas.



8.2 Conceptos de la arquitectura Java: JDK, JRE, JVM, SE, EE, ME

Es fundamental diferenciar entre tanta cantidad de siglas que podemos encontrar rodeando al mundo de la programación, y más concretamente al mundo de Java.



JDK (Java Development Kit): es un entorno de desarrollo de software utilizado para desarrollar aplicaciones y applets de Java. Incluye Java Runtime Environment (JRE), un intérprete/cargador (Java), un compilador (javac), un archivador (jar), un generador de documentación (Javadoc) y otras herramientas necesarias para el desarrollo de Java.

JRE: significa “Java Runtime Environment”. Proporciona los requisitos mínimos para ejecutar una aplicación Java; consiste en la Java Virtual Machine (JVM), clases principales y archivos auxiliares.

Además, existen unas plataformas especiales para soporte del lenguaje que son:

- **Java SE:** Para escritorio y *applets*.
- **Java EE:** Para el diseño y desarrollo de aplicaciones web (servidores) y de empresa.
- **Java ME:** para otro tipo de dispositivos “empotrados” como electrodomésticos, vehículos, etcétera...

8.3 Curiosidades sobre Java

- El sistema operativo para móviles “Android” está programado, fundamentalmente, en Java.
- Java fue creado en 1996 para la empresa Sun Microsystems. Una empresa que luego pasaría a ser propiedad de ORACLE (conocida por su potente sistema gestor de bases de datos).
- Java es uno de los lenguajes de programación más populares para aplicaciones de cliente-servidor de web, con más de diez millones de usuarios registrados.

8.4 Tipos de aplicaciones en Java

La versatilidad del lenguaje de programación Java permite al programador crear distintos tipos de aplicaciones. A continuación, describiremos las características más relevantes de cada uno de ellos:

- **Aplicaciones de consola:**
 - Son programas independientes al igual que los creados con los lenguajes tradicionales.
 - Se componen como mínimo de un archivo .class que debe contar necesariamente con el método main.
 - No necesitan un navegador web y se ejecutan cuando invocamos el comando Java para iniciar la Máquina Virtual de Java (JVM). De no encontrarse el método main la aplicación no podrá ejecutarse.
 - Las aplicaciones de consola leen y escriben hacia y desde la entrada y salida estándar, sin ninguna interfaz gráfica de usuario.

- **Aplicaciones gráficas:**
 - Aquellas que utilizan las clases con capacidades gráficas, como Swing que es la biblioteca para la interfaz gráfica de usuario avanzada de la plataforma Java SE.
 - Incluyen las instrucciones import, que indican al compilador de Java que las clases del paquete Javax.swing se incluyan en la compilación.
- **Applets:**
 - Son programas incrustados en otras aplicaciones, normalmente una página web que se muestra en un navegador. Cuando el navegador carga una web que contiene un applet, éste se descarga en el navegador web y comienza a ejecutarse. Esto nos permite crear programas que cualquier usuario puede ejecutar con tan solo cargar la página web en su navegador.
 - Se pueden descargar de Internet y se observan en un navegador. Los applets se descargan junto con una página HTML desde un servidor web y se ejecutan en la máquina cliente.
 - No tienen acceso a partes sensibles (por ejemplo: no pueden escribir archivos), a menos que uno mismo le dé los permisos necesarios en el sistema.
 - No tienen un método principal.
 - Son multiplataforma y pueden ejecutarse en cualquier navegador que soporte Java.
- **Servlets:**
 - Son componentes de la parte del servidor de Java EE, encargados de generar respuestas a las peticiones recibidas de los clientes.
 - Los servlets, al contrario de los applets, son programas que están pensados para trabajar en el lado del servidor y desarrollar aplicaciones Web que interactúen con los clientes.
- **Midlets:**
 - Son aplicaciones creadas en Java para su ejecución en sistemas de propósito simple o dispositivos móviles. Los juegos Java creados para teléfonos móviles son midlets.
 - Son programas creados para dispositivos embebidos (se dedican a una sola actividad), más específicamente para la máquina virtual Java MicroEdition (Java ME).
 - Generalmente son juegos y aplicaciones que se ejecutan en teléfonos móviles.